

Attention 与 Transformer

从内容路由到 FlashAttention：同一数学语义的高效实现

技术快照：2026-08-22

[返回 Topic Landing Page](#)

Contents

1 本册定位：Attention 是信息路由机制，Transformer 是架构组合	3
2 Attention 的对象：Query 提问，Key 匹配，Value 提供内容	4
2.1 贯穿母例：同一个 query 怎样把路由权重变成输出	5
2.2 为什么需要缩放因子	6
3 Self、Cross、Causal：改变的是信息可达关系	6
3.1 Self-Attention	6
3.2 Cross-Attention	6
3.3 Causal Attention	7
4 Multi-Head Attention：并行建立多个路由子空间	7
5 位置信息：纯内容路由本身不知道顺序从哪里来	8
6 Transformer block：把路由与逐位置变换交替堆叠	8
7 Attention 扩展的两个轴：序列长度与模型深度不能混为一谈	9
7.1 FlashAttention 与 Linear Attention：一个优化实现，一个改变映射	9
7.2 Kimi Delta Attention：有限状态记忆中的衰减、纠正与写入	10
7.3 从 Gated DeltaNet 到 KDA 再到 Gated DeltaNet-2：有限状态记忆到底要控制什么	11
7.4 Hybrid Attention：Linear 与 Global Attention 可以共存	11
7.5 Attention Residuals：路由发生在 depth，而不是 token sequence	12
7.6 从 Block AttnRes 到 Delta AttnRes：下一步研究的是“应该路由哪种深度表示”	12
7.7 Kimi K3：同一个大模型同时优化 sequence、depth 与 channel 三条信息流	13
8 长序列的第二个问题：数学正确不等于硬件高效	15
9 Stable Softmax：分块之前先解决数值稳定性	17
10 Online Softmax：把“必须看完整行”改写成可合并状态	18
11 Blockwise Online Softmax：FlashAttention 的状态不变量	18
12 Causal tiling：能在 tile 层跳过的工作，不要等到元素层再 mask	21
13 GPU 实现最小模型：软件任务层级与硬件执行资源要分开	21

14 Triton 的 blocked-program 抽象：先描述数据块，再让编译器映射线程	22
14.1 为什么不能把 <code>tl.load</code> 定义成“HBM 到 shared memory”	23
14.2 Autotune：搜索的是资源交换，不是神奇常数	23
15 Backward：本册只组合 Attention 的 VJP，不重讲通用 AutoDiff	24
16 为什么 forward 保存 LogSumExp，而不是保存完整概率矩阵	25
17 Backward 的循环为什么要按目标梯度重新分组	26
18 正确性与性能边界：必须同时回答“算得对”和“为什么快”	27
18.1 数学正确性	27
18.2 性能不变量不是“shared memory 永远更快”	27
19 FlashAttention 的版本演进：优化对象会随硬件改变	27
20 实际工程调用：会用高性能 Attention 不等于必须手写 kernel	28
21 最小调用接口：看到什么问题先回到哪一层	28
22 一页压缩：从内容路由重建整套 Attention	29
23 资料与版本边界	30

1 本册定位：Attention 是信息路由机制，Transformer 是架构组合

序列建模的核心困难之一是：一个位置当前需要的信息可能来自很远的位置，而且“该看谁”并不是固定的。卷积用固定局部窗口汇总邻域，循环网络把历史压入递归状态；Attention 采用另一条路线：让当前位置根据内容产生查询，并对候选信息动态分配权重。

Mother Question | 怎样按内容动态路由信息

给定一组表示，怎样让每个 query 位置根据自己的当前内容，对一组 key 计算相关性，再用这些相关性汇总对应 value；随后又怎样把这种路由机制与残差、归一化、前馈网络和位置信息组合成可堆叠的 Transformer？

这先确定 Attention/Transformer 的数学与架构语义。只有在这套语义固定以后，才进入第二条由实现瓶颈触发的分支：当序列很长时，怎样在保持标准 Attention 数学语义的同时，避免大型中间矩阵反复写入和读取 device global memory（高端加速器上通常由 HBM 承载）？FlashAttention 回答的是这个实现问题，而不是重新定义 Attention。

本册有三层责任：

1. 语义层: Attention 为什么需要 Q, K, V , 权重怎样产生, self/cross/causal/multi-head 分别改变什么;
2. 架构层: Attention 怎样进入 Transformer block, 以及 positional information 为什么不可缺;
3. 实现层: 朴素 Attention 的数据流为什么昂贵, online softmax 怎样使 tiled exact attention 成为可能, FlashAttention 与 Triton 又如何把数学结构映射到 GPU。

通用 reverse-mode AD、JVP/VJP 不在本册重新定义; 本册只调用 Area 50 AutoDiff 给出的 VJP 接口, 并负责 Attention 自己的 backward 组合。

概念 A	≠	概念 B	真正区别与题目信号	混淆后果
Attention	≠	Transformer	Attention 是动态信息路由算子; Transformer 是围绕 attention、FFN、residual、normalization、position 等构造的架构族	会把一个算子误当成完整网络架构
Transformer	≠	LLM	Transformer 是 architecture family; LLM 还包含训练目标、数据、优化、tokenization、decoding 与系统实现	会把“使用 Transformer”误当成“已经定义语言模型生命周期”
FlashAttention	≠	Approximate Attention	FlashAttention 重新安排精确 Attention 的计算与数据搬运; 它不靠低秩/稀疏近似改变标准输出	会把性能优化误解成数学近似
FlashAttention	≠	KV Cache	FlashAttention 优化一次 attention kernel 的计算/内存流; KV cache 复用自回归推理中历史 token 的 K/V	会把训练/长序列 kernel 优化和 decode 状态复用混为一谈

2 Attention 的对象: Query 提问, Key 匹配, Value 提供内容

设 query 序列长度为 N_q , key/value 序列长度为 N_k 。一个 attention head 中:

$$Q \in \mathbb{R}^{N_q \times d_k}, \quad K \in \mathbb{R}^{N_k \times d_k}, \quad V \in \mathbb{R}^{N_k \times d_v}.$$

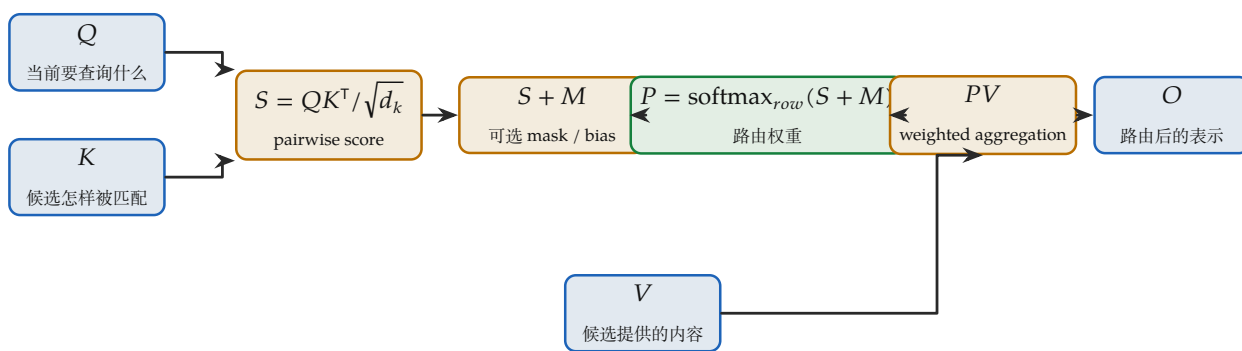
Scaled dot-product attention 定义为

$$S := \frac{QK^T}{\sqrt{d_k}},$$

$$P := \text{softmax}_{\text{row}}(S + M),$$

$$O = PV.$$

这里 M 表示可选的 mask / additive bias。若某个位置不允许被关注，可以在 softmax 前把对应 score 设为 $-\infty$ ，使其指数权重为零。



位置只表示数据依赖顺序；框之间的几何距离不表示计算耗时或重要性。

Figure 1 Scaled dot-product Attention 的信息流。箭头表示张量数据依赖； QK^T 产生路由分数，row-wise softmax 把每行变成归一化权重，再用权重汇总 V 。

Q, K, V 的角色不要靠字母死记。对第 i 个 query 与第 j 个 candidate key:

$$S_{ij} = \frac{q_i^T k_j}{\sqrt{d_k}}.$$

S_{ij} 回答“第 i 个位置当前的查询与第 j 个位置提供的匹配索引有多相容”。Softmax 把同一 query 对所有 key 的相对分数变成一组权重；真正被汇总的内容是 v_j 。

因此可以把一行输出写成

$$o_i = \sum_{j=1}^{N_k} P_{ij} v_j.$$

这说明 Attention 是一种内容依赖的加权路由：权重由 Q, K 决定，被路由的载荷由 V 提供。

2.1 贯穿母例：同一个 query 怎样把路由权重变成输出

考虑一个 query 对三个 key 的缩放后 score:

$$s = (1, 0, 2),$$

并令三个 value 为

$$v_1 = (1, 0), \quad v_2 = (0, 1), \quad v_3 = (1, 1).$$

以最大 score 2 为指数基准，softmax 权重与

$$(e^{-1}, e^{-2}, 1)$$

成比例，因此

$$o = \frac{e^{-1}v_1 + e^{-2}v_2 + v_3}{1 + e^{-1} + e^{-2}} = \frac{(1 + e^{-1}, 1 + e^{-2})}{1 + e^{-1} + e^{-2}}.$$

这个例子后面会再次出现。先把它看成普通 Attention：score 决定三条信息边的相对权重，输出只是对应 value 的加权和。进入 FlashAttention 后，数学目标不变，只改变这三个权重与加权和的计算顺序。

2.2 为什么需要缩放因子

若 q_r, k_r 的各维分量近似独立、均值为零、方差量级相近，则点积

$$q^T k = \sum_{r=1}^{d_k} q_r k_r$$

的方差会随 d_k 增长。较大的 logits 会把 softmax 推入更尖锐的区域，使梯度尺度更难控制。除以 $\sqrt{d_k}$ 的作用是让 score 的典型尺度不随 head dimension 线性扩张。

这是尺度控制，不意味着所有真实训练分布都严格满足独立同分布假设；该推导解释设计动机，而不是对任意模型激活分布的精确统计定律。

3 Self、Cross、Causal：改变的是信息可达关系

3.1 Self-Attention

Self-attention 中 Q, K, V 都来自同一组输入表示的不同投影。于是每个位置可以根据当前内容重新读取同一序列中的其他位置。

3.2 Cross-Attention

Cross-attention 中 query 与 key/value 来自不同表示集合，例如 decoder state 查询 encoder output。此时一般有

$$N_q \neq N_k,$$

所以输出 shape 跟 query 侧一致：

$$O \in \mathbb{R}^{N_q \times d_v}.$$

3.3 Causal Attention

自回归场景要求位置 i 不能读取未来位置 $j > i$ 。定义

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

于是

$$P_{ij} = 0 \quad (j > i).$$

Causal mask 改变的是允许的信息边，不是 softmax 本身的数学定义。

4 Multi-Head Attention：并行建立多个路由子空间

若模型维度为 d_{model} ，多头注意力通过不同参数投影得到多组

$$Q^{(h)}, K^{(h)}, V^{(h)},$$

分别执行 attention：

$$\text{head}_h = \text{Attention}(Q^{(h)}, K^{(h)}, V^{(h)}).$$

然后拼接并线性投影：

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O.$$

多头并不保证“每个 head 必然学一种人类可解释关系”。更准确地说，模型获得多组独立参数化的相似度与 value 投影通道，可以并行形成不同的信息路由模式。

5 位置信息：纯内容路由本身不知道顺序从哪里来

如果同时对输入位置做同一个置换，而没有额外 positional signal，self-attention 会相应置换输出；它本身不会凭空知道“第 3 个 token”和“第 30 个 token”的绝对或相对顺序。

因此，如果任务不仅需要 causal mask 这类“谁能看谁”的可达性约束，还需要表达 token 的绝对位置、相对距离或顺序关系，就要通过某种 positional mechanism 把这些信息注入表示或 attention score。具体实现可以是绝对位置编码、相对位置 bias、旋转位置表示等；这些实现会演化，但它们都在补充纯内容相似度本身没有提供的位置关系：

$$\text{content identity} + \text{position relation} \rightarrow \text{order-aware routing}$$

这里的箭头表示信息组合，不表示数学蕴含。

6 Transformer block：把路由与逐位置变换交替堆叠

Attention 负责跨位置交换信息，feed-forward network (FFN) 负责对每个位置的 channel representation 做非线性变换。Residual connection 为原表示提供直接通路；normalization 控制子层输入或输出的尺度，并改变深层网络的优化动态。具体效果取决于 norm 类型与放置位置，因此这里只保留架构职责，不把某一种 placement 当成唯一标准。

现代 Transformer 存在 Pre-Norm、Post-Norm、RMSNorm 等多种变体，因此不能把某一种 norm placement 写成“Transformer 的定义”。下面只用常见 Pre-Norm 形式展示信息流：

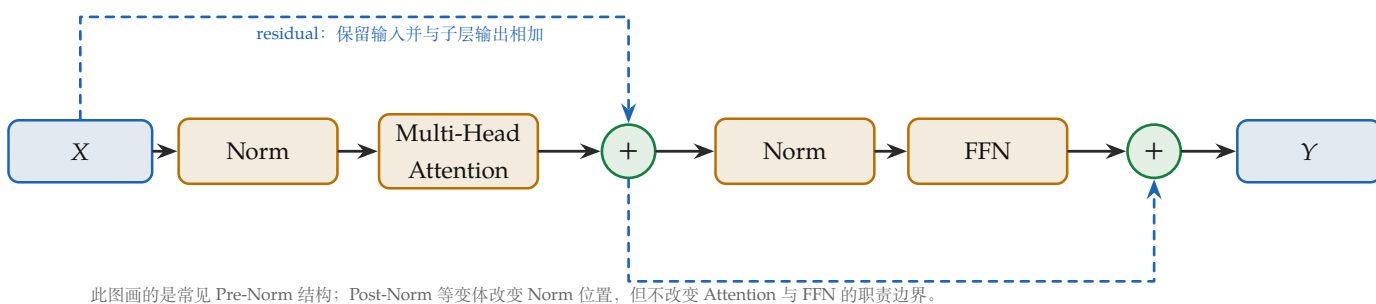


Figure 2 一个常见 Pre-Norm Transformer block 的信息流。实线箭头表示前向数据流，绕行边表示 residual addition；其他 norm placement 仍属于同一 architecture family。

抽象地看，一层不断交替执行两件事：

1. 跨位置路由：当前位置从其他位置读取什么；

2. 逐位置计算：读取后的表示在 feature/channel 维度上怎样变换。

所以 Transformer 不是 “Attention 连续堆叠”。没有 FFN、residual、normalization 与 position，不能把单独 attention 算子直接等同完整 Transformer block。

7 Attention 扩展的两个轴：序列长度与模型深度不能混为一谈

到这里需要先把一个容易被新模型术语混淆的边界钉死：“让更长序列算得更便宜” 和 “让更深网络选择更高层表示” 是两个不同问题。它们都可能使用 attention / routing 语言，但路由发生的轴不同。

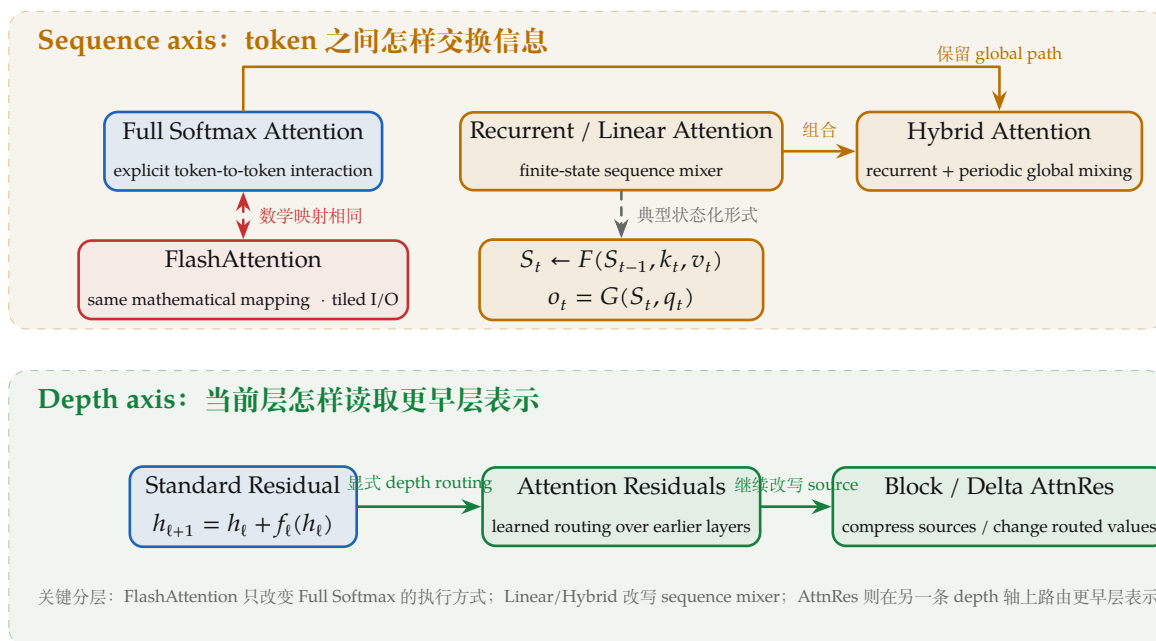


Figure 3 Attention 相关扩展的两个架构轴与一个实现层。上半部分沿 sequence 轴区分 Full Softmax、recurrent/linear 与 hybrid mixer; FlashAttention 单独挂在 Full Softmax 下方, 表示“数学映射相同、执行方式不同”。下半部分沿 depth 轴讨论 residual / AttnRes 对更高层表示的聚合。

7.1 FlashAttention 与 Linear Attention：一个优化实现，一个改变映射

标准 causal Softmax Attention 的第 t 个输出会显式依赖此前所有 key/value:

$$o_t = \sum_{j \leq t} \text{softmax}(s_t)_j v_j.$$

FlashAttention 仍然计算这一个映射，只是通过 tiling、online softmax 和片上复用避免物化完整 S/P 。因此它仍然属于**精确 Softmax Attention 的 I/O-aware 实现**，核心序列计算没有从二次 all-pairs interaction 变成线性 recurrence。

Recurrent / linear attention 走的是另一条路：不再让当前 query 与全部历史 key 显式形成完整 score row，而是把过去压缩进固定大小的 recurrent state。抽象形式可以写成

$$S_t = F(S_{t-1}, k_t, v_t), \quad o_t = G(S_t, q_t).$$

这样每一步读取的是状态 S_t ，而不是长度随历史增长的完整 KV 集合。代价是：它已经改变了 token mixing 的数学结构。因此它不能被描述为“更快的 FlashAttention”，也不能默认与标准 Softmax Attention 数值等价。

7.2 Kimi Delta Attention：有限状态记忆中的衰减、纠正与写入

Kimi Delta Attention (KDA) 是当前 recurrent linear attention 的一个具体例子。它从 Gated DeltaNet 发展而来，用 fine-grained decay gate 控制固定大小的矩阵状态。Moonshot 当前公开的 KDA forward 将每个 head 的状态写成

$$S_t = (I - \beta_t k_t k_t^T) \text{Diag}(\alpha_t) S_{t-1} + \beta_t k_t v_t^T$$

并读取

$$o_t = S_t^T q_t.$$

这里 $S_t \in \mathbb{R}^{d_k \times d_v}$ 的 shape 不随序列长度增长； $\alpha_t \in (0, 1)^{d_k}$ 是逐 channel 的 retention / decay gate， β_t 控制当前 delta-rule correction / write 的强度。把更新拆开看更容易理解：

1. 先用 α_t 对旧状态做选择性衰减；
2. 再沿当前 key 方向修正旧关联，避免简单叠加不断污染状态；
3. 最后写入当前 $k_t v_t^T$ 关联；
4. query q_t 从更新后的有限状态中读取输出。

因此把 KDA 说成“滚动平均”会丢掉最关键的结构：它维护的是 key-value associative matrix state，并带有衰减、delta correction 与新关联写入，而不是维护一个标量均值。

有限状态也带来新的 trade-off。历史被压缩后，系统不再拥有标准 Softmax Attention 那种对任意过去 token 的显式独立访问；需要通过 state update 学会保留、覆盖和读取有用关联。换来的则是更稳定的长序列状态规模与更低的 decode cache 压力。

7.3 从 Gated DeltaNet 到 KDA 再到 Gated DeltaNet-2：有限状态记忆到底要控制什么

把近期 linear-attention 进展放在一起看，真正稳定的研究问题不是“谁把 $O(N^2)$ 变成 $O(N)$ ”，而是：当全部历史被压缩进有限状态以后，模型怎样精确决定保留、擦除和写入什么？

可以把一个 recurrent memory update 抽象成三个相互关联但不完全相同的控制量：

1. **Retention / Decay**：旧状态哪些 channel 继续保留，哪些应该快速衰减；
2. **Erase / Correction**：当前 key 指向的旧关联是否需要先被纠正或擦除；
3. **Write**：新的 key-value 关联以多大强度写入状态。

Gated DeltaNet 把 adaptive forgetting 与 delta-rule correction 结合起来；KDA 的关键推进是把 decay 从更粗粒度控制扩展为逐 channel 的 fine-grained decay，从而更细致地管理有限状态容量。2026 年的 Gated DeltaNet-2 又进一步指出：erase 与 write 本身仍然不必绑定在同一个门控强度上，于是把两者解耦为独立控制。

因此这条演进最值得保留的不是三个模型名，而是一个更一般的有限记忆设计原则：

compressed recurrent memory $\Rightarrow$ forget / erase / write 是需要分别建模的控制问题

这不是“所有 linear attention 都必须采用三个独立 gate”的定理；它表达的是设计诊断：若这些作用长期被同一个粗粒度控制量绑定，模型可能难以同时处理过度遗忘、旧关联污染和新信息写入不足。

Gated DeltaNet-2 的当前结果只来自作者报告的有限模型规模与训练设置；它说明“解耦 erase/write”是有实验证据的研究方向，但还不能据此宣布它已经取代 KDA 或 full attention。

7.4 Hybrid Attention：Linear 与 Global Attention 可以共存

Linear attention 与 full/global attention 也不是必须二选一。Kimi Linear 的公开架构采用 3:1 的 KDA-to-global MLA 层比例：多数层利用 recurrent state 提高长上下文效率，周期性 global attention 层保留更直接的全局 token interaction。

这给出一个可复用的设计视角：

cheap recurrent mixing + occasional global mixing

是否值得这样混合，取决于质量、长上下文需求、KV-cache 成本、训练并行化和目标硬件；“linear”并不意味着所有 full attention 都应被删除。

7.5 Attention Residuals: 路由发生在 depth, 而不是 token sequence

标准 Pre-Norm residual stack 会把各层产生的更新沿深度方向持续累加。展开多层以后, 可以把它理解为更早层贡献以固定加法权重进入后续 hidden state。Attention Residuals (AttnRes) 改变的是这一跨层聚合规则: 当前层不再只接受固定单位权重的累计结果, 而是通过 learned softmax attention 在多个较早 layer outputs 之间选择性聚合。

因此这里的 “attention” 回答的是:

> 当前第 ℓ 层应该从哪些较早深度的表示读取多少信息?

而普通 self-attention 回答的是:

> 当前 token 应该从哪些 token positions 读取多少信息?

把 Full AttnRes 抽象化, 可以写成:

$$\pi_{\ell,j} = \text{softmax}_j(s_{\ell,j}), \quad r_{\ell} = \sum_{j<\ell} \pi_{\ell,j} h_j,$$

其中 h_j 是更早深度的候选表示, $s_{\ell,j}$ 是当前层对这些 depth sources 的 learned score。标准 residual 相当于把历史贡献通过固定加法不断累积; AttnRes 则把 “哪些早期表示值得继续进入当前层” 变成 input-dependent routing。

两者一个沿 **depth axis** 路由, 一个沿 **sequence axis** 路由。Full AttnRes 若直接保存并访问所有早期 layer outputs, 会带来随深度增长的 activation memory 与 pipeline communication。Block AttnRes 因此把若干相邻层压成 block-level sources: 当前层只在少量 block representations 之间做 depth-wise routing, 换取更低的状态与通信开销。

7.6 从 Block AttnRes 到 Delta AttnRes: 下一步研究的是 “应该路由哪种深度表示”

AttnRes 把 “跨层路由” 显式化以后, 又暴露出新的设计自由度: 被路由的 value 到底应该是累计 hidden state, 还是每一层真正新增的信息?

2026 年后续工作 Delta Attention Residuals 提出使用

$$\Delta h_j = h_{j+1} - h_j$$

作为 depth-routing 的候选表示, 而不是直接在高度相关的累计 h_j 之间选择。作者的解释是: 深层 cumulative states 往往彼此高度冗余, 使 softmax routing 越来越接近低对比度; layer delta 更直接表示 “这一层新写入了什么”。其在作者报告的 220M-7.6B 模型上优于标准 residual 与原始 AttnRes。

这里必须保持证据边界: Delta AttnRes 是独立后续研究, 不是 Kimi K3 的已发布结构; 当前结果说明 “depth routing 的 source representation 选择本身是一阶设计问题”, 而不是证明所有大模型都应把

AttnRes 改成 delta routing。

概念 A	≠	概念 B	真正区别与题目信号	混淆后果
FlashAttention	≠	KDA / Linear Attention	前者保持标准 Softmax Attention 映射并优化 I/O；后者用 recurrent finite state 改写 sequence mixer	会把 exact kernel optimization 与 sequence-mixer replacement 混成一件事
Token Attention	≠	Attention Residuals	前者沿 sequence positions 路由；后者沿 model depth 在较早 layer representations 之间路由	会误以为 AttnRes 是另一种长上下文 token attention
MoE Routing	≠	Attention Routing	MoE 决定一个 token 进入哪些 expert/FFN 参数子网络；Attention 决定表示从哪些 token 或 layer source 聚合信息	会把“只激活少量 experts”误解成 attention 变稀疏

7.7 Kimi K3：同一个大模型同时优化 sequence、depth 与 channel 三条信息流

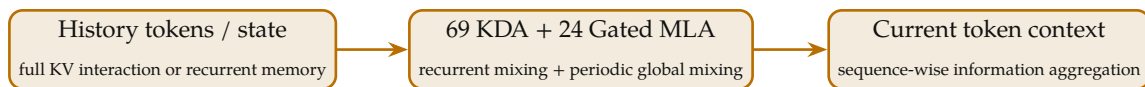
Kimi K3 值得作为架构阅读案例，不是因为“2.8T 参数很大”，而是因为它把三类本来容易混在一起的路由问题同时放进一个模型：

轴	K3 中的机制	真正解决的问题
Sequence mixing	69 KDA + 24 Gated MLA layers	历史 token 怎样以有限状态高效压缩，同时周期性保留 global content interaction
Depth mixing	Block Attention Residuals	当前层怎样选择性读取较早 depth representations，缓解固定 residual accumulation 的 dilution
Channel / parameter mixing	Stable LatentMoE, 896 routed experts 中每 token 选择 16 个，另有 shared experts	怎样把参数容量扩得很大，但只让少量 expert 子网络参与当前 token 计算

信息来源

路由 / 混合机制

送往哪里

Sequence axis**Depth axis****Channel / parameter axis**

Kimi K3 · 2026 architecture snapshot: 2.8T total · 104B activated · 93 layers · 1,048,576 context

三行共享同一读法：先找“从哪里读”，再看“怎样路由/混合”，最后看“写到哪里”。三个轴可以同时存在，但路由对象完全不同。

Figure 4 Kimi K3 的三轴分解。Sequence mixing、depth mixing 与 expert/channel mixing 是三个正交程度较高的架构问题；同一 block 可以同时包含这些机制，但一个轴的“稀疏/路由”不能拿来解释另一个轴。

当前官方配置是 93 layers、2.8T total parameters、约 104B activated parameters、1,048,576 context length。69 个 KDA 与 24 个 Gated MLA 形成约 3:1 的 sequence-mixing hybrid；这比“全部改成 linear attention”更能说明当前工程取向：用多数便宜 recurrent layers 承担长序列状态更新，再周期性插入 global layers 补充显式全局交互。

Stable LatentMoE 又解决完全不同的 channel/parameter 问题。按 K3 当前公开结构，可以把 routed branch 压成：

$$z = W_{\text{down}}x \in \mathbb{R}^{3584},$$

$$u = \sum_{i \in \mathcal{J}(x)} p_i E_i(z), \quad |\mathcal{J}(x)| = 16,$$

$$y_{\text{routed}} = W_{\text{up}} \text{RMSNorm}(u) \in \mathbb{R}^{7168}.$$

Router 仍根据 full-width representation 选择 896 routed experts 中的 Top-16，但真正 routed-expert computation 在较窄的 latent width 3584 中进行，再投影回 residual width 7168。这样做的系统意义是：参

数容量可以通过大量 experts 扩展，而每 token 的 expert weight traffic、expert compute 与 distributed dispatch payload 不必按 full hidden width 同比例增长。

在高稀疏 MoE 中，另一个一阶问题是负载均衡：若少数 experts 长期过热，理论稀疏性并不会自动转化为训练吞吐。K3 的 Stable LatentMoE 使用 Quantile Balancing，根据 router-score quantiles 调整 expert allocation bias；其目的不是改变最终 semantic routing 定义，而是让极高 expert sparsity 下的训练负载更稳定。K3 技术报告所称相对 K2 约 2.5× 的 overall scaling-efficiency improvement 是整套架构、训练和数据 recipe 的作者报告结果，不能单独归因于 KDA、AttnRes 或 LatentMoE 中某一个模块。

因此，读类似新模型时最稳的拆法不是背产品参数，而是依次问：

1. sequence 方向的信息如何混合；
2. depth 方向的信息如何保留与选择；
3. channel / parameter 容量如何激活；
4. 这些数学结构怎样映射到 kernel、通信、cache 与并行系统。

MoE 的一般路由、负载均衡与分布式 expert parallel 不是本 Attention Topic 的 Canonical Owner，这里只保留理解 K3 组合架构所需的接口。

8 长序列的第二个问题：数学正确不等于硬件高效

对 self-attention，若

$$Q, K, V \in \mathbb{R}^{N \times d},$$

则

$$S = QK^T \in \mathbb{R}^{N \times N},$$

以及

$$P = \text{softmax}(S) \in \mathbb{R}^{N \times N}.$$

标准 attention 的核心计算仍具有二次序列规模：

$$\Theta(N^2d).$$

朴素 kernel chain 还可能把 S 、 P 分别写入 device global memory；在 HBM-backed GPU 上，这对应把大型中间矩阵写入 HBM，再由后续 kernel 重新读回：



因此运行时间不仅由 FLOPs 决定，还受 data movement 限制。可以用 arithmetic intensity 建立最小性能语言：

$$AI := \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

当 memory traffic 相对于计算量过大时，增加更多算术单元并不会自动线性加速 kernel。

FlashAttention 的根本转换

FlashAttention 不改变标准 Attention 的数学映射，也没有把 $\Theta(N^2)$ attention 工作变成线性计算。它改变的是执行顺序：把 Q, K, V 分块，在片上尽可能完成 score、softmax 状态更新和 value accumulation，避免把完整 $N \times N$ 的 S 、 P 反复物化到 device global memory。浮点运算次序可能与朴素实现不同，因此这里的“相同”指数学语义相同，而不是逐 bit 相同。

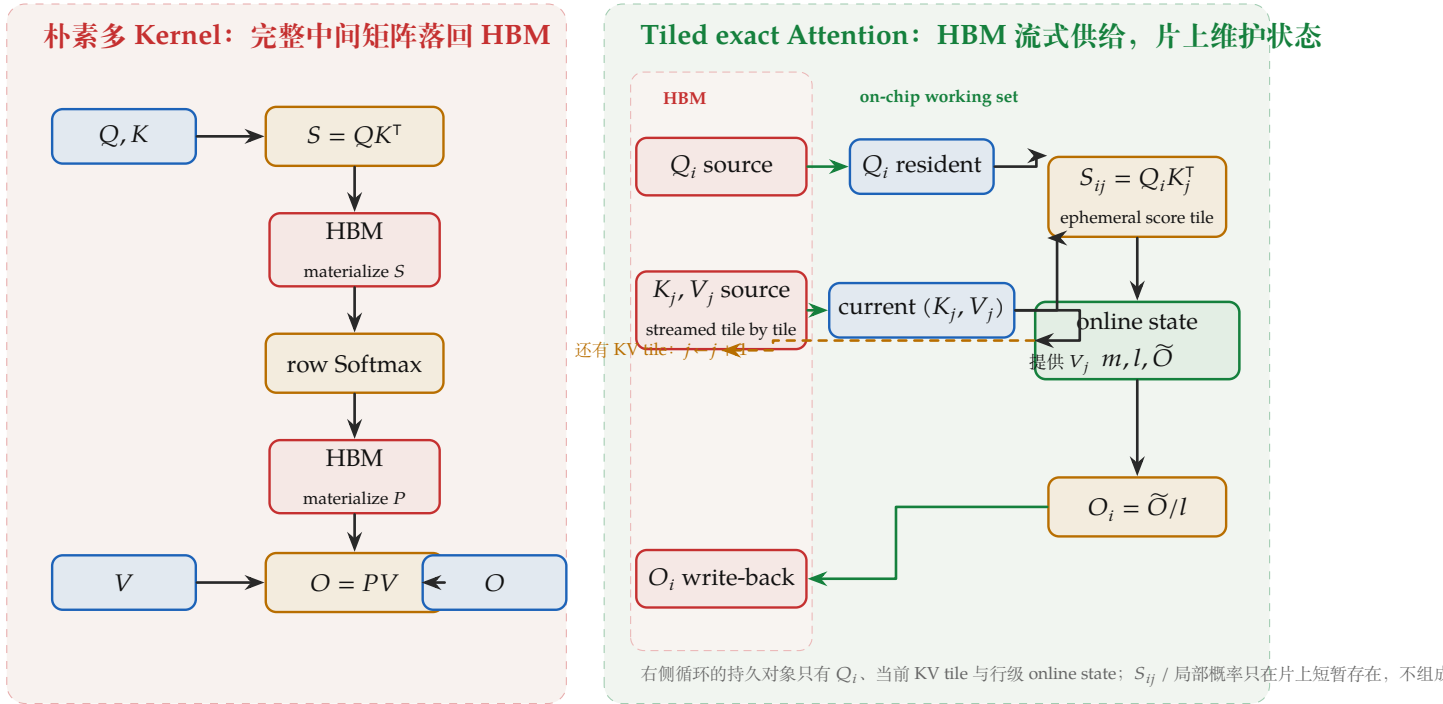


Figure 5 朴素 Attention 与 tiled exact Attention 的数据流对比。左侧多次把完整中间矩阵写入 HBM；右侧固定一个 query tile，流式读取 KV tiles，并只维护行级统计量与输出累加器。图只表达数据流与驻留层次，不表示真实延迟比例。

9 Stable Softmax：分块之前先解决数值稳定性

对一行 logits x ，softmax 为

$$p_i = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

直接指数化可能 overflow。利用 softmax 的平移不变性，令

$$m := \max_j x_j,$$

则

$$p_i = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}.$$

此时

$$x_i - m \leq 0,$$

所以指数项不超过 1。减最大值改变中间表示，却不改变最终归一化概率。

10 Online Softmax：把“必须看完整行”改写成可合并状态

普通 stable softmax 似乎要求先看到整行，才能知道全局最大值。若 attention score 被拆成多个 tile，这会成为关键障碍。

先看一维流式版本。处理到前 t 个元素时，定义

$$m_t := \max_{1 \leq j \leq t} x_j,$$

$$l_t := \sum_{j=1}^t e^{x_j - m_t}.$$

加入新元素 x_{t+1} 后：

$$m_{t+1} = \max(m_t, x_{t+1}).$$

旧的指数和原本以 m_t 为基准，必须换到新的基准 m_{t+1} 。因为

$$e^{x_j - m_{t+1}} = e^{x_j - m_t} e^{m_t - m_{t+1}},$$

所以

$$l_{t+1} = l_t e^{m_t - m_{t+1}} + e^{x_{t+1} - m_{t+1}}.$$

这条递推的关键不只是“少一次遍历”，而是 m, l 已经包含继续合并后续数据所需的全部行级信息。新数据到来时，不需要保存前面所有 logits。

11 Blockwise Online Softmax：FlashAttention 的状态不变量

固定一个 query tile Q_i ，依次读入 key/value tile (K_j, V_j) 。当前 score tile：

$$S_j = \frac{Q_i K_j^\top}{\sqrt{d}}.$$

对 query tile 中每一行维护三个状态：

$$m, \quad l, \quad \tilde{O}.$$

它们的精确定义是：对目前已经处理过的 key 集合 $\mathcal{J}$ ，第 r 行满足

$$m_r = \max_{j \in \mathcal{J}} S_{rj},$$

$$l_r = \sum_{j \in \mathcal{J}} e^{S_{rj} - m_r},$$

$$\tilde{o}_r = \sum_{j \in \mathcal{J}} e^{S_{rj} - m_r} v_j.$$

因此对已经处理的部分，当前精确归一化输出就是

$$o_r = \frac{\tilde{o}_r}{l_r}.$$

这个定义就是 FlashAttention forward 的核心不变量。

读到新 tile 后，先计算

$$m_{\text{new}} = \max(m_{\text{old}}, \text{rowmax}(S_j)),$$

以及旧状态的缩放系数

$$\alpha = e^{m_{\text{old}} - m_{\text{new}}}.$$

当前未归一化权重为

$$P_j^* = e^{S_j - m_{\text{new}}}[:, \text{None}].$$

于是更新：

$$l_{\text{new}} = \alpha \odot l_{\text{old}} + \text{rowsum}(P_j^*)$$

和

$$\tilde{O}_{\text{new}} = \alpha[:, \text{None}] \odot \tilde{O}_{\text{old}} + P_j^* V_j.$$

所有 KV tiles 处理完成后：

$$O_i = \tilde{O} / l[:, \text{None}].$$

为什么更新以后仍然精确

新最大值只改变指数的公共参考点。把旧 l 与旧 $\tilde{O}$ 同时乘以 $e^{m_{\text{old}} - m_{\text{new}}}$ ，等价于把所有旧项重新表达在新的指数基准下；随后再加入新 tile 的项。因此每一步都保持上面的三条状态定义，最终结果与整行一次性 stable softmax 完全相同。

这也是理解 FlashAttention 时最值得记住的三元组：

$$(m, l, \tilde{O}).$$

回到前面的母例，把前两个 score (1,0) 作为第一个 tile，把 score 2 作为第二个 tile。处理完第一个 tile：

$$m_{\text{old}} = 1, \quad l_{\text{old}} = 1 + e^{-1}, \quad \tilde{o}_{\text{old}} = (1, e^{-1}).$$

第二个 tile 到来后，新最大值变成 2，所以

$$\alpha = e^{1-2} = e^{-1},$$

而新 tile 的未归一化权重就是 1。于是

$$l_{\text{new}} = e^{-1}(1 + e^{-1}) + 1 = 1 + e^{-1} + e^{-2},$$

$$\tilde{o}_{\text{new}} = e^{-1}(1, e^{-1}) + (1, 1) = (1 + e^{-1}, 1 + e^{-2}).$$

最终 $\tilde{o}_{\text{new}} / l_{\text{new}}$ 与前面整行一次性 softmax 得到的输出完全相同。这个数值轨迹直接展示了为什么旧 tile 必须在最大值变化时同时重标定 l 和 $\tilde{O}$ 。

12 Causal tiling：能在 tile 层跳过的工作，不要等到元素层再 mask

在 causal self-attention 中，score matrix 的上三角无效。若 query/key tile 对齐，那么对某个 query tile：

- 对角线左侧的完整 KV tiles 全部合法；
- 对角线 tile 内需要 element-wise causal mask；
- 对角线右侧的完整 KV tiles 全部非法，可以直接不计算。

因此高效 kernel 常把 causal 区域拆成“完整有效 tile”与“对角边界 tile”。这里体现的是一种常见优化思路：若粗粒度结构已经能证明一整块工作无贡献，就不要先算完再逐元素归零。

13 GPU 实现最小模型：软件任务层级与硬件执行资源要分开

为了理解 FlashAttention kernel，只需要掌握一套最小 CUDA execution model，而不需要把本册变成 CUDA 教材。

- kernel launch 产生一个 grid；
- grid 由 thread blocks 组成；
- NVIDIA GPU 上一个 warp 通常由 32 个 threads 组成；
- thread block 被调度到 streaming multiprocessor (SM) 执行；
- 一个 SM 在资源允许时可以同时驻留多个 blocks / warps；
- registers、shared memory/L1、L2、device global memory 构成不同作用域和代价的存储层级；在高端数据中心 GPU 上，device global memory 常由 HBM 承载。

因此必须阻断三个错误等号：

thread $\neq$ physical CUDA core,

warp $\neq$ thread block,

SM $\neq$ thread block.

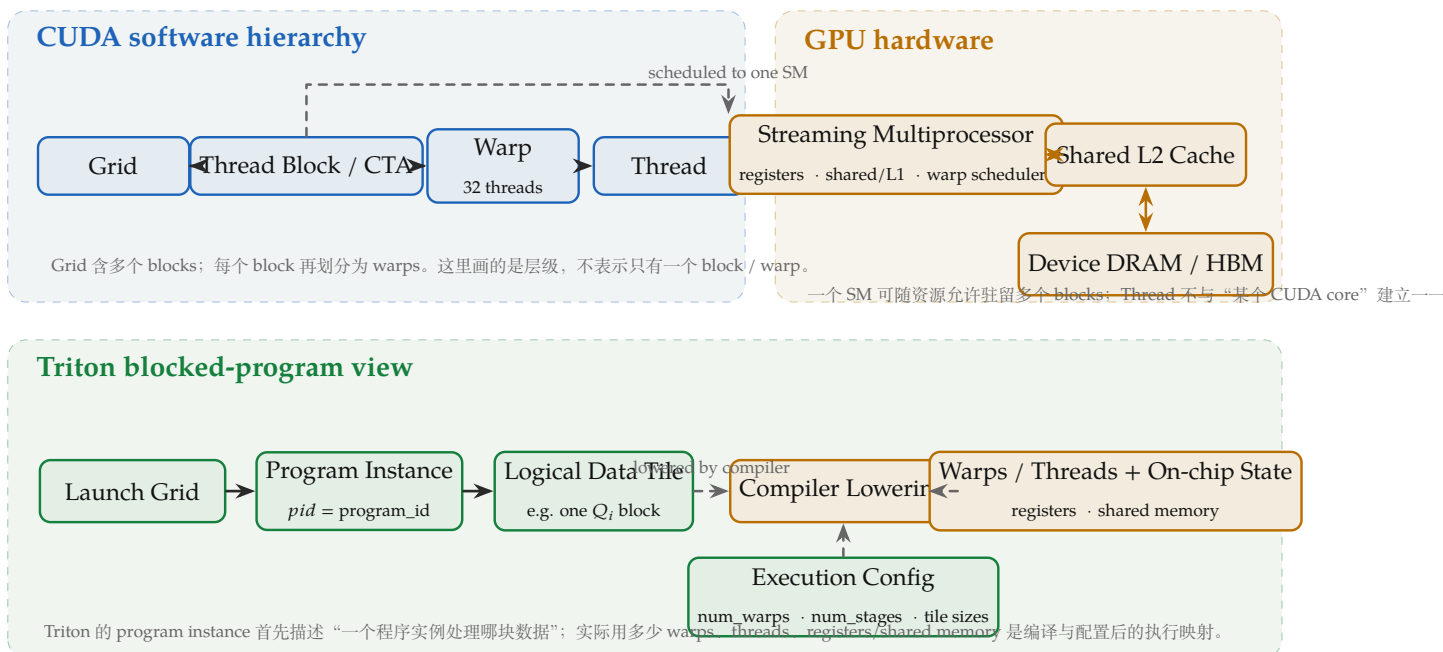


Figure 6 CUDA 与 Triton 的抽象层映射。上半部分把 CUDA software hierarchy 与 GPU hardware hierarchy 分开：Thread Block / CTA 被调度到某个 SM，但两者不是同一对象；下半部分展示 Triton 的 program instance / data tile 经编译器与 execution config 映射到 warps、threads 和片上状态。虚线表示实现映射，不表示对象相等。

同一 warp 内发生 data-dependent branch divergence 时，硬件需要分别执行实际被采用的路径，并对当前路径不参与的 lanes 做 inactive masking；因此 divergence 影响吞吐。现代 NVIDIA GPU 支持 Independent Thread Scheduling，所以不能把“warp 内线程永远逐指令锁步”当成程序正确性的永久假设。

14 Triton 的 blocked-program 抽象：先描述数据块，再让编译器映射线程

CUDA C 常从“每个 thread 负责哪个 scalar/work item”开始；Triton 更强调一个 program instance 对一个 tensor block 的运算。

典型一维结构是：

```
1 pid = tl.program_id(0)
2 offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
3 mask = offsets < n
4 x = tl.load(x_ptr + offsets, mask=mask)
```

这里

BLOCK_SIZE

描述的是 program 要处理的数据范围，不意味着“一定存在同样数量的物理 threads”。执行线程数还受编译配置与 backend 映射影响。

对 FlashAttention，一个自然 program grid 是让 program instance 对应

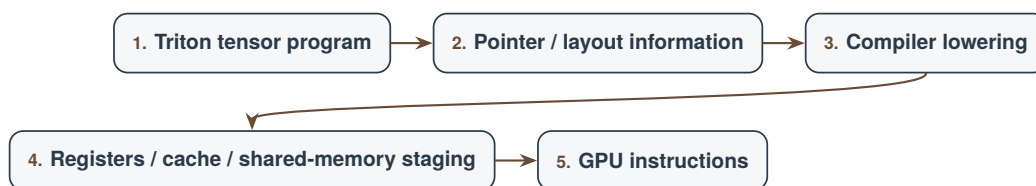
$(batch, head, query\ tile),$

然后固定该 query tile，在 kernel 内循环读取 KV tiles。这样不同 query tiles 可以并行，而每个 program 内部又能让当前 Q 与运行状态在片上复用。

14.1 为什么不能把 `tl.load` 定义成“HBM 到 shared memory”

`tl.load` 的语言语义是从 pointer 指定的位置加载 tensor values。编译器最终可能利用 registers、cache、shared memory、异步数据搬运等不同机制；这些属于 lowering 与目标硬件实现。

因此更安全的模型是：



这条链中的箭头表示实现层映射，不表示每次 load 必然依次经过所有列出的硬件层。

14.2 Autotune：搜索的是资源交换，不是神奇常数

Triton 的 autotune 允许程序员给出候选配置，例如 tile size、num_warps、num_stages，并指定 tuning key。对新的 key 组合，运行时评测候选配置并选择更快者。

num_warps 表示每个 kernel instance 协同执行所使用的 warps 数；在 NVIDIA backend 上，8 warps 对应 $8 \times 32 = 256$ threads。num_stages 指编译器在适用循环上进行 software pipelining 时采用的 stages 数；具体怎样 lower 到异步搬运、寄存器或 shared-memory staging，仍取决于 compiler/backend 与目标硬件。

增加 tile、warps 或 stages 可能提高数据复用与 memory/compute overlap，但也可能增加 registers、shared memory 占用并降低 occupancy。因此 autotuning 的目标是特定 workload + hardware 上的资源平衡，而不是发现一个跨设备永远最优的常数。

15 Backward: 本册只组合 Attention 的 VJP, 不重讲通用 AutoDiff

设

$$S = sQK^\top, \quad s := \frac{1}{\sqrt{d}},$$

$$P = \text{softmax}(S),$$

$$O = PV.$$

为避免把“微分”与“loss 对张量的梯度”混成同一记号, 下文用上横线表示 reverse-mode cotangent:

$$\bar{O} := \frac{\partial L}{\partial O}.$$

工程代码里常把这些量命名为 dO、dQ、dK、dV; 这里的数学正文统一写成 $\bar{O}, \bar{Q}, \bar{K}, \bar{V}$ 。Area 50 AutoDiff 已经拥有 matmul VJP 与 softmax VJP, 本册只把它们按 Attention 的 computation graph 组合。

首先:

$$\boxed{\bar{V} = P^\top \bar{O}}$$

以及

$$\boxed{\bar{P} = \bar{O}V^\top}.$$

对每一 softmax row, 定义

$$D_i := \sum_j P_{ij} \bar{P}_{ij}.$$

则 softmax VJP 为

$$\boxed{\bar{S} = P \odot (\bar{P} - D[:, \text{None}])}.$$

最后

$$\boxed{\bar{Q} = s \bar{S} K}$$

以及

$$\bar{K} = s \bar{S}^T Q.$$

D_i 还可以改写成

$$D_i = \sum_k O_{ik} \bar{O}_{ik}.$$

因为

$$\sum_j P_{ij} \bar{P}_{ij} = \sum_j P_{ij} \sum_k \bar{O}_{ik} V_{jk} = \sum_k \bar{O}_{ik} O_{ik}.$$

因此 backward 可以先做一次 row-wise preprocess:

$$D = \text{rowsum}(O \odot \bar{O}).$$

16 为什么 forward 保存 LogSumExp，而不是保存完整概率矩阵

Forward 已经计算过每行的 running maximum m_i 与 denominator l_i 。定义

$$LSE_i := m_i + \ln l_i.$$

因为

$$P_{ij} = \frac{e^{S_{ij}-m_i}}{l_i} = e^{S_{ij}-m_i-\ln l_i},$$

所以

$$P_{ij} = e^{S_{ij}-LSE_i}.$$

这意味着 backward 不需要保存完整 $N \times N$ 的 P 。只要保留 backward 所需的输入/输出与每个 query row 的少量统计量，就可以在当前 tile 上重新计算 score 和 probability，立即用于 $\bar{V}, \bar{S}, \bar{Q}, \bar{K}$ ，然后丢弃。具体需要保存哪些张量取决于实现接口，不能把某一版 kernel 的保存集合写成普遍定义。

这是一种明确的 compute-memory trade-off:

少保存大型 activation + backward 重算局部 tile

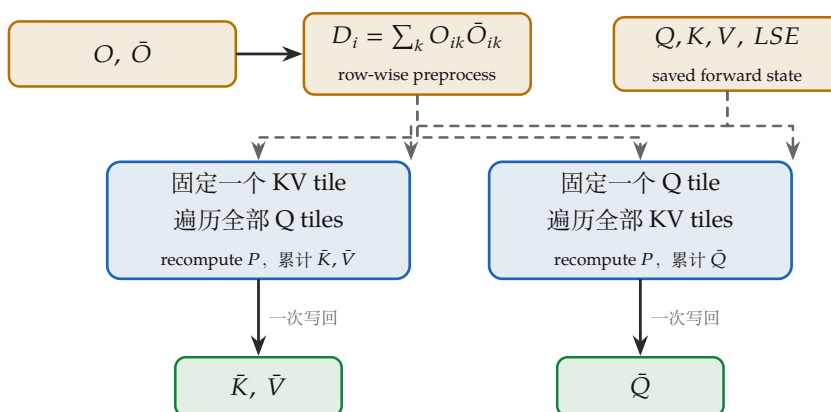
如果重算所增加的 compute 比大型中间矩阵的 HBM write/read 更便宜，这种交换既能降低峰值显存，也可能提升实际速度。

17 Backward 的循环为什么要按目标梯度重新分组

每个 $\bar{Q}_i$ 依赖所有 KV tiles；每个 $\bar{K}_j, \bar{V}_j$ 又依赖所有 Q tiles。若在双重循环中不断更新所有部分梯度，会造成重复 device-memory 写回或复杂同步。

更好的组织方式是：

1. 先预计算 $D_i = o_i^T \bar{o}_i$ ；
2. 固定一个 KV tile，遍历所有 Q tiles，在片上累计该 tile 的 $\bar{K}, \bar{V}$ ，最后写回；
3. 固定一个 Q tile，遍历所有 KV tiles，在片上累计该 tile 的 $\bar{Q}$ ，最后写回。



两条 kernel 路径使用同一梯度公式，只改变累积顺序；目标是让当前梯度 tile 尽量留在片上直到完成。

Figure 7 FlashAttention backward 的两种累积方向。两条主路径共享前置的 D 与 LSE/recomputed P ，但分别固定 KV tile 或 Q tile，使目标梯度尽可能在片上完成累计后再写回。

这个循环重排不是新的梯度公式；它是在保持相同数学依赖的前提下，选择更适合 GPU data locality 的执行顺序。

18 正确性与性能边界：必须同时回答“算得对”和“为什么快”

18.1 数学正确性

FlashAttention forward 的数学正确性来自 $(m, l, \tilde{O})$ 不变量；backward 的正确性来自对同一个标准 Attention 映射应用 matmul 与 softmax 的 VJP，并在需要时重算与 forward 一致的 masked probability。由于浮点运算次序不同，优化实现不要求与朴素实现逐 bit 相同，而应在明确的 dtype 与容差下验证数值一致性。

因此验证至少分两层：

- **numerical correctness**：与高精度/框架 reference 的 output、 $\bar{Q}, \bar{K}, \bar{V}$ 在明确且与 dtype 相匹配的浮点容差内一致；
- **structural correctness**：causal mask、scale、shape、stride、边界 tile 与 dtype 路径必须和 reference 语义一致。

18.2 性能不变量不是“shared memory 永远更快”

FlashAttention 的性能优势依赖硬件、shape、dtype、kernel schedule 与实现版本。可以稳定保留的判断只有：

- 避免大型中间矩阵反复写入/读出 HBM 能减少 data movement；
- tile 提高数据复用，但 tile 过大可能造成 register/shared-memory pressure；
- software pipelining 可以 overlap memory 与 compute，但增加在途状态；
- 更高 occupancy 不等于更高性能，Tensor Core utilization、bandwidth、instruction mix 同样重要；
- 最优配置依赖目标 GPU 与 workload，不能从单次实验或单一设备的结果固化为永久常数。

19 FlashAttention 的版本演进：优化对象会随硬件改变

FlashAttention 1 把 attention 作为 I/O-aware exact algorithm 重新设计：用 tiling、online softmax 与 recomputation 减少 HBM traffic，同时保持标准 Attention 数学语义。

FlashAttention-2 进一步指出，仅减少 I/O 还不够。它减少 non-matmul FLOPs，提高 sequence 方向的并行度，并重新划分 thread block / warp 工作，以降低 shared-memory 读写和低 occupancy 带来的损失。

FlashAttention-3 面向 Hopper，进一步利用 Tensor Memory Accelerator (TMA)、Tensor Core 的异步能力和 warp specialization，让数据搬运、矩阵乘法与 softmax 更充分重叠，并加入面向 FP8 的低精度路径。

截至 2026-08-22，官方 FlashAttention 仓库还提供 FlashAttention-4：它使用 CuTeDSL 实现，并针对 Hopper 与 Blackwell GPU 优化。FA4 的具体算子覆盖、dtype、shape 与硬件支持仍在持续演进，因此不能把“FA4”理解成一个对所有设备和所有 Attention 变体都固定可用的永久接口。

这些版本变化说明：数学映射可以保持不变，但 kernel 的瓶颈会随着硬件执行资源、数据搬运能力和编译器抽象改变。可以长期保留的设计原则是：

硬件—算法共同设计

先找当前硬件上最昂贵的资源：是 HBM traffic、shared-memory traffic、non-matmul instruction、Tensor Core feed、exp throughput 还是 synchronization；再重新安排 tile、流水线和工作划分。硬件的相对成本一变，最优 kernel 结构也可能随之改变。

20 实际工程调用：会用高性能 Attention 不等于必须手写 kernel

如果目标只是训练或推理一个普通 Transformer，优先使用框架已经提供的 scaled dot-product attention / fused backend，让框架根据设备、dtype、shape 和支持条件选择实现。

手写 Triton Attention 更适合以下目的：

- 学习 GPU kernel 与 data movement；
- 研究新的 attention variant / mask / bias / layout；
- 现有 fused backend 无法表达当前算子语义；
- 对目标硬件做明确、可测量的性能研究。

否则自定义 kernel 还要承担 correctness、backward、dtype、mask、stride、corner case、benchmark、未来硬件兼容等维护成本。

21 最小调用接口：看到什么问题先回到哪一层

当问题出现 Attention 相关信号时，不要一上来就背公式。先判断当前问题属于哪一层：

问题层	第一观察对象	典型问题
语义层	Q, K, V 与允许的信息边	self/cross/causal 到底改变什么；某个位置能读谁
架构层	Attention 与 FFN/residual/norm/position 的组合	为什么 Attention 不等于 Transformer；位置信息从哪里进入
序列扩展层	full token interaction、recurrent state、hybrid mixer	当前是在优化标准 Softmax，还是已经改写 sequence mixer；KDA 为什么不等于 FlashAttention
深度扩展层	residual accumulation 与 cross-layer routing	AttnRes 在 layer/depth 之间选表示，为什么不等于 token attention
数值层	score range 与 row-wise normalization	为什么减最大值；online softmax 为什么仍精确
实现层	中间矩阵、tile、memory traffic	为什么 FlashAttention 快；哪些对象不再物化到 HBM
反向层	computation graph 与 saved/recomputed state	$\bar{Q}, \bar{K}, \bar{V}$ 从哪来；为什么保存 LSE 而不保存 P
性能层	workload + hardware resource balance	block size、warps、stages 为什么需要调优

若问题的核心已经变成 generic JVP/VJP、reverse mode 或 custom autograd contract，就停止在本册继续扩展，转交 Area 50 AutoDiff。

22 一页压缩：从内容路由重建整套 Attention

一句话压缩

Attention 的核心是“按内容选择信息来源”：标准 Softmax Attention 在 token sequence 上显式建立路由，Transformer 再把这种跨位置路由与 FFN、residual、normalization 和 position 组合；长序列可以选择保持映射不变的 FlashAttention，也可以用 recurrent/linear/hybrid attention 改写 sequence mixer；AttnRes 处理的是另一条 depth 轴上的跨层表示路由，而 MoE 又在 channel/parameter 轴上选择参数子网络。现代大模型可以同时优化这三条轴。

重建本册只需要回答十四个问题：

1. Q, K, V 分别控制“提问、匹配、载荷”的哪一部分？
2. $1/\sqrt{d}$ 为什么需要控制 score scale？
3. self、cross、causal attention 改变的是哪一种信息可达关系？
4. multi-head 与 positional information 分别补了什么能力？

5. Attention 为什么不等于 Transformer?
6. FlashAttention 与 recurrent/linear attention 的根本边界是什么?
7. KDA 为什么需要有限状态的 decay、delta correction 与 KV write, 而不能理解成普通滚动平均?
8. 从 Gated DeltaNet 到 KDA / Gated DeltaNet-2, forget、erase、write 为什么逐渐被拆成更细控制量?
9. token-sequence routing 与 AttnRes 的 depth-wise routing 为什么是两个不同轴?
10. Delta AttnRes 为什么改为路由 layer delta, 而不是累计 hidden state? 这个结论的证据边界在哪里?
11. Kimi K3 的 sequence、depth、channel 三条优化轴分别由什么机制负责, 为什么不能互相解释?
12. 朴素 Attention 为什么会制造昂贵的 $N \times N$ memory traffic?
13. $(m, l, \tilde{O})$ 三个状态分别保存什么不变量?
14. 为什么 LSE 足以让 backward 重算 P ?
15. 为什么 kernel 的最佳 tile/warp/pipeline 配置必须跟 workload 和硬件一起看?

23 资料与版本边界

本册的稳定数学与架构主线依据 Transformer、Online Softmax、FlashAttention、FlashAttention-2、FlashAttention-3、Kimi Linear 与 Attention Residuals 的原始论文; Gated DeltaNet-2 与 Delta Attention Residuals 作为 2026 年后续研究, 用来说明 finite-state memory control 和 depth-routing source representation 仍在快速演化, 不作为已定型结论。FlashAttention-4 与 KDA 的当前实现状态分别以官方 FlashAttention / MoonshotAI 实现为准。Kimi K3 只作为 2026 年组合 KDA、AttnRes 与 Stable LatentMoE 的当前架构实例, 不承担这些机制的定义。GPU 执行语义按 NVIDIA CUDA Programming Guide 核验; Triton program、autotune、num_warps、num_stages 与 fused-attention 行为按 2026-08-22 的 Triton 官方文档核验。具体框架 API、backend 支持、dtype/shape 覆盖、模型参数与性能数字均属于版本敏感信息, 使用时应重新核对当前官方文档。

- Triton Config / autotune: [triton.Config](#), [triton.autotune](#);
- Triton fused attention tutorial: [Fused Attention](#);
- FlashAttention 官方仓库: [Dao-AILab/flash-attention](#);
- Kimi Linear: [Kimi Linear paper](#), [MoonshotAI/Kimi-Linear](#);
- KDA 当前参考实现: [MoonshotAI/minitriton KDA forward](#);
- Attention Residuals: [Attention Residuals](#);
- Gated DeltaNet-2: [Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention](#);

- Delta Attention Residuals: [Delta Attention Residuals](#);
- Kimi K3 技术报告: [Kimi K3: Open Frontier Intelligence](#), [MoonshotAI/Kimi-K3](#)。